

ONLINE JUDGE SYSTEM

在线评测系统 实验报告

课程大作业 2 · FastAPI + Async SQLAlchemy + Streamlit

姓名	
学号	
班级	
仓库	<code>github.com/OverMoon64/online-judge-system</code>

可运行 · 可测试 · 可复现 · 面向课程本地验收

摘要

本项目实现了一个面向课程本地验收的在线评测系统。后端采用 FastAPI、SQLAlchemy Async 与 SQLite，前端采用 Streamlit；系统提供用户与角色管理、题目管理、动态语言配置、异步代码评测、提交检索与重测、公开日志与访问审计，以及基于 OpenAI-compatible Chat Completions 协议的可取消 AI 智能命题流程。

所有后端路由均为异步接口，响应统一为 {code, msg, data}，并返回真实 HTTP 状态码。判题器在独立临时目录中以参数数组启动编译器或解释器，同时限制时间、进程树内存与输出大小，支持 AC、WA、TLE、MLE、RE、CE、UNK。

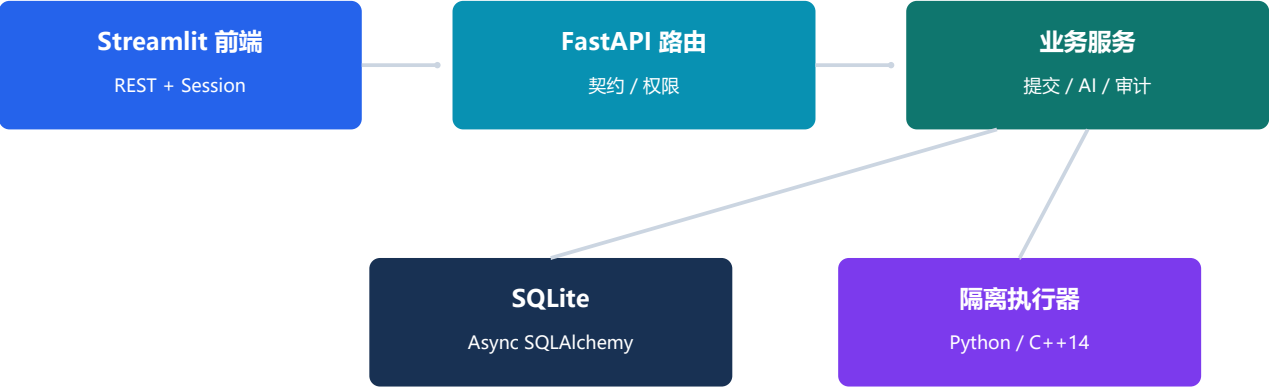
评分覆盖

模块	分值	实现要点
基础模块	30	认证、题目、语言、异步判题、提交、日志、审计与重置
AI 智能命题	10	兼容模型、阶段进度、取消、费用、双解校准与有效性检测
代码规范	5	分层异步架构、Ruff、测试与 GitHub Actions
实验报告	5	架构、评分点、关键实现、测试、安全边界与验收流程

说明：本系统定位为课程本地验收项目，不把进程级限制宣称为可直接暴露公网的强隔离沙箱。

1 系统架构

系统采用浏览器、REST API、业务服务、异步数据库与隔离执行器五层结构。Streamlit 不直接读取 SQLite，所有权限与数据约束均由 FastAPI 后端统一裁决。



分层职责

层	职责	关键技术
界面	账户、题目、提交、日志、AI 页面	Streamlit + HTTP Session
接口	校验、认证、错误映射、分页	FastAPI + Pydantic
业务	判题编排、统计、审计、AI 任务	asyncio + httpx
数据	事务、关系约束、初始化与重置	SQLAlchemy Async + SQLite
执行	编译运行、资源监控、清理	async subprocess + psutil

工程目录

app/ 后端核心 · frontend/ Streamlit · tests/ 自动化测试 · scripts/ 演示与报告 · .vscode/ WSL 开发配置

2 API 契约与权限

成功和失败响应均使用 {code, msg, data}。FastAPI 默认校验错误被转换为 400；组合失败条件严格执行 401 > 403 > 400 > 429 > 409 > 404 > 500，保证边界请求也具有确定行为。

认证与角色

角色	可执行操作	限制
user	浏览/新增/修改题目、注册语言、提交代码	只能查看本人提交，受日志可见性和限流约束
admin	题目/语言/角色管理、全量提交、审计	仍经过统一数据校验
banned	无	旧会话也在依赖层被拒绝

会话与密码

登录后由后端签发随机 Session Token，签名 Cookie 设置为 HttpOnly，数据库保存当前有效 Token；退出时服务端撤销 Token 并清除 Cookie，因此旧签名 Cookie 无法重放。密码使用 bcrypt 哈希，且用户可以修改自己的密码。启动和重置均确保存在 admin / admintestpassword；每次请求重新读取用户角色，因此角色变更和封禁立即生效。

日志访问矩阵

场景	返回内容	是否审计
管理员	完整 details + 总分	是
已登录用户 + public_cases=True	完整 details + 总分	是
提交者 + public_cases=False	仅 score / counts	是
其他用户 + public_cases=False	403	是（资源存在）
未登录 / 参数错误 / 资源不存在	401 / 400 / 404	否

3 异步判题器

提交保存为 pending 后立即响应，后台任务再负责编译和逐点运行。该设计把 HTTP 延迟与用户程序耗时解耦，也使前端可以安全轮询进度。



后台任务立即接管；超时、超内存或取消时终止整个进程组并清理临时目录

安全执行链

措施	实现细节	防护目标
命令白名单	校验可执行程序、占位符与参数；不用 shell=True	命令注入
独立目录	每次评测使用临时工作目录	文件冲突与残留
进程组	新会话启动，结束时杀死完整进程树	孤儿进程
三类限制	硬超时、递归 RSS、输出上限	TLE/MLE/输出洪泛
最终清理	等待回收并删除临时目录	资源泄漏

结果判定

编译失败为 CE；运行异常为 RE；超过时间/内存分别为 TLE/MLE；退出正常后进行文本比较得到 AC/WA；无法分类的内部异常为 UNK。每个 AC 测试点 10 分。

输出比较

逐行移除行末空格并忽略最终多余换行，但保留前导空格。该规则容忍常见排版差异，同时避免把有意义的缩进误判为相同。

4 提交、统计与生命周期

提交列表支持用户、题目、状态筛选和分页。普通用户只能看到本人数据，管理员可以查看全部。限流按用户滚动一分钟计数，每分钟最多 3 次，并依照课程要求先于资源不存在检查。

重新评测

管理员对同一提交 ID 发起重测，系统清除旧结果并重新进入后台队列，而不是复制一条新提交。这保持了资源标识稳定，也便于验收时确认“同 ID 重测”。

一致性设计

需求	实现
提交总数	按用户计数全部提交
通过题数	仅统计 AC，并按 problem_id 去重
删除题目	事务内级联删除 testcase、submission、judge_log、access_log，并清理文件点
系统重置	先取消判题/AI 任务，再重建数据与管理员
退出会话	重置响应同时清除当前 Session

状态与故障恢复

业务状态写入数据库；内存中的后台任务注册表只负责运行期取消。服务重启不会暴露密钥，也不会把未完成任务误标为成功。内部异常通过统一处理器返回 500，避免泄露堆栈。

5 AI 智能命题

模型配置采用 OpenAI-compatible Chat Completions 协议，支持 Qwen、DeepSeek、Kimi 及其他兼容服务。每位用户可保存多个 URL、模型名、API Key、输入/输出单价和推理强度配置。API Key 不进入数据库、日志或响应，而是使用由 OJ_SESSION_SECRET 派生的密钥加密保存到本地配置文件，因此服务重启后仍可使用。

分阶段 workflow

阶段	进度	产物 / 校验
需求分析	10%	难度、知识点、约束与歧义分析
题面与双语解法	45%	完整题面、Python/C++ 参考代码和初始输入
交叉运行与答案校准	75%	双解逐点共识，校准错误或空白答案
定向修复	80-94%	仅在必要时携带诊断反馈，最多重试两次
边界与规模有效性	96-100%	确定性大数据文件点、试运行与有效性评分

Token 与费用

系统优先采用提供商返回的 usage，分别展示输入、输出、推理、缓存和总 Token；缺少总量时按已知分项归一计算，并标明估算。费用依据用户手工配置的输入/输出单价和计价单位换算为人民币，保留到分，不自动抓取易变化的厂商价格。所有任务用量会累加到对应模型配置。

可靠性与取消

模型输出先抽取 JSON 并归一化常见字段，再经过 Pydantic 严格结构校验。C++ 参考程序只编译一次，Python 与 C++ 对全部输入交叉运行；存在安全锚点且逐点一致时，系统以共识输出校准模型手算错误。仅单语言失败时进行定向修复；双解冲突、无锚点或资源异常时禁止静默回填，最多携带诊断重试两次。取消会传播到 HTTP 流式请求和本地试运行。

测试用例有效性

AI 题目在生成成功后尝试构造确定性的最小边界点和最大规模文件点，覆盖标量、数组、字符串、网格与图等常见输入结构。系统通过 Python/C++ 共识生成 .in/.out，并给出 0-100 的有效性评分与逐项说明；无法可靠识别格式时安全降级，不阻断题目生成，也不额外消耗模型 Token。

人工闸门：生成完成后展示题面、资源限制、双语解法、测试点、答案校准和有效性明细；登录用户确认后才载入新增题目表单，作者固定为实际模型名称。

6 Streamlit 前端

前端采用顶部中文导航，按账户、题库与评测、提交记录、AI 智能命题和后台管理组织。题目详情与代码编辑器位于同一页，提交记录和详情独立分页。浏览器 Cookie 被加密保存并绑定当前 Streamlit 服务进程：完整刷新可恢复，服务重启会拒绝并删除旧载荷。统一 API 客户端始终使用同一 Base URL；所有操作仅调用 REST API，权限最终仍由后端判断。

页面覆盖

页面组	核心交互
账户中心	顶部显示用户名与角色；注册、登录、退出、资料和修改密码
题库与评测	题库点击详情；同页展示题面、限制、IDE 编辑器和独立提交按钮
提交记录	每页 5 条；筛选、详情、源代码、测试点日志与管理员重测
AI 命题	模型选择、自动难度、推理强度、进度、取消、Token/费用与人工导入
后台管理	用户角色、语言配置、访问审计、数据重置

异步反馈

提交详情使用局部刷新显示 pending/running 到最终状态，AC、WA、TLE、MLE、RE、CE 和 UNK 使用一致的彩色状态标记。私有日志只显示 score/counts，公开日志或管理员才显示逐点结果。所有 API 错误统一展示 HTTP 状态、code 与 msg；401 会清理失效会话。完整刷新可恢复有效会话，服务重启不恢复旧账号；退出会调用后端 logout、撤销服务端 Token 并清理本地 Cookie。Streamlit 以 headless 模式监听 127.0.0.1，不依赖 WSL Interoperability 自动打开浏览器。

界面验收截图



图 1 顶部导航与 AI 智能命题页面（自动难度与测试点策略）

7 测试与质量保障

测试分为纯接口测试、Linux 真实判题集成测试、AI 假提供商测试和人工浏览器流程。测试数据库逐用例隔离，避免状态相互污染。

自动化结果

Ubuntu 22.04、Python 3.10.12、G++ 11.4.0；Ruff 与格式检查通过；pytest 79 passed；覆盖率 89.57%（门槛 85%）；GitHub Actions 成功。

测试域	代表场景
契约	字段校验、统一响应、422→400、错误优先级
认证	Cookie 跨 rerun/完整刷新、服务重启失效、退出清理、旧 Cookie 拒绝
判题	Python/C++: AC、WA、RE、CE、TLE、MLE、空白比较；AC 耗时不超限
数据	分页筛选、限流、去重统计、同 ID 重测、重置
审计	私有日志仅总分、公开/管理员完整明细、403 与访问记录矩阵
AI	usage/费用、取消、双解校准、文件压力点、有效性评分与安全降级

持续集成

GitHub Actions 在 Ubuntu 和 Python 3.10 上安装 G++，运行 Ruff 静态检查、格式检查与 pytest，并设置 85% 覆盖率门槛。VS Code 配置对应的 WSL 解释器、后端/前端任务、调试入口和测试入口；前端任务固定使用 headless 模式。

8 安全边界与改进方向

当前实现通过命令白名单、无 Shell 执行、临时目录、进程组和资源限制降低本地评测风险，足以服务课程验收；但进程级措施并不是公网多租户环境所需的强隔离。

若部署到公网

- 将评测迁移到一次性容器或专用沙箱；使用非特权用户、只读根文件系统和网络命名空间。
- 使用 cgroup v2、seccomp 和严格挂载策略，把 CPU、内存、进程数与系统调用限制交给内核。
- 引入消息队列和独立 Judge Worker，增加幂等任务、重试、崩溃恢复和横向扩展。
- 把 SQLite 替换为生产数据库，将审计日志持久化，并配置 HTTPS、CSRF 与速率限制网关。

隐私与密钥

API Key 不写数据库、日志、Git 或响应，而是加密保存到本地忽略文件；.env、数据库、临时文件与缓存均纳入 .gitignore。示例配置不包含真实秘密。

AI 使用说明

本项目使用 AI 辅助需求梳理、代码实现、测试设计与报告排版。所有内容均通过静态检查、自动化测试或人工审阅验证；AI 生成题目必须经使用者预览后才导入题库。

9 结论

本实验把异步 Web API、数据库、操作系统进程管理、权限审计和大模型工作流整合为一个完整系统。实现重点不仅是运行代码，更是在明确 API 契约下正确处理并发、资源回收、权限优先级、数据一致性与失败路径。

项目提供 VS Code Remote WSL 配置、演示种子数据、验收清单、自动化测试和 CI，使开发、验证与演示均可重复执行。AI 命题通过双语言共识校准、确定性边界/规模点、有效性评分和人工闸门，在成功率、区分度与可审阅性之间取得平衡。

交付物

交付	位置
源代码	GitHub: OverMoon64/online-judge-system
运行指南	README.md
验收流程	docs/demo-checklist.md
报告源稿	docs/report.md
PDF 报告	output/pdf/online-judge-system-report.pdf

END OF REPORT